

Model Context Protocol (MCP) – Deep Research Analysis

Executive Summary: The Model Context Protocol (MCP) is an open standard (introduced by Anthropic in Nov 2024 [83†L21-L25]) for connecting AI models/agents with external tools, data sources and workflows. It is often called the “USB-C for AI” because it provides a single, JSON-RPC-based interface for AI assistants to discover and invoke diverse capabilities (e.g. file systems, databases, APIs) [83†L21-L25] [45†L98-L102] . MCP uses a client-server architecture with JSON-RPC 2.0 over STDIO or HTTP/SSE transport [17†L133-L142] [45†L139-L146] , and defines primitives like **Tools**, **Resources**, and **Prompts** to structure interactions [46†L209-L218] [45†L139-L146] . The ecosystem has grown rapidly – by 2026 there are reportedly thousands of MCP servers and native support in major platforms (Claude, ChatGPT/Agent Plugins, etc.) [83†L54-L63] [63†L56-L64] . MCP’s spec has evolved through several versions (initial v1 in 2024, major updates in Nov 2025 and July 2026) that added features and made the protocol stateless for better scale [83†L21-L25] [12†L28-L33] . This report covers MCP’s specifications, reference implementations, architecture, example workflows (with diagrams), API formats, integration patterns, security/scalability considerations, interoperability with related standards, tooling, real-world cases, limitations, and next steps (including a minimal demo plan with code).

Official Specification and Version History

Anthropic open-sourced MCP in November 2024 as a “new standard for connecting AI assistants to the systems where data lives” [83†L21-L25] . The initial spec v1.0 (dated Nov 25, 2024) defined the JSON-RPC protocol, message formats, authentication using OAuth 2.1, and basic primitives for two-way “plug-in” style communication [83†L21-L25] [46†L209-L218] . By late 2025, the spec had been updated (Nov 2025 release) to support **multi-step tasks**, richer security features, and enterprise-grade controls [13†L50-L58] [83†L54-L63] . A major July 2026 update (version 2026-07-28) removed the stateful session handshake entirely, making MCP stateless to improve scalability [12†L28-L33] [11†L33-L41] . In practice this means each request now includes version/method metadata in headers (e.g. `MCP-Method`, `MCP-Version`) and any server replica can handle it [11†L33-L41] [12†L28-L33] . Throughout its evolution, MCP has been developed as an open GitHub project (hosted by the Linux Foundation) under an MIT license [72†L224-L227] [69†L228-L232] .

- **Key Timeline:**

- *Nov 2024:* MCP v1.0 released by Anthropic (open-sourced, MIT license) [83†L21-L25] .
- *Nov 2025:* v2.0 adds task workflows, simplified auth flows, and enterprise features [13†L50-L58] .
- *Jul 2026:* v3.0 makes MCP stateless (no session ID/handshake) and refines header-based routing [12†L28-L33] [11†L33-L41] .

Repositories, SDKs, and Implementations

The official **ModelContextProtocol** GitHub organization contains the specification, docs, and reference code [72†L174-L178] [72†L182-L193] . It maintains multi-language **SDKs** (all MIT-licensed): e.g. TypeScript, Python, Java, Kotlin, C#, Go, PHP, Ruby, Rust, Swift [72†L182-L193] [69†L228-L232] . Star counts (2026) illustrate popularity: the Python SDK ($\approx 24.2k$ ★) and TypeScript SDK ($\approx 13.3k$ ★) are most used, with C# ($\sim 4.5k$ ★), Rust ($\sim 3.9k$ ★) and others also maintained [73†L241-L249] [73†L253-L260] . The MCP org also provides a **“servers”** repository (90.2k★) listing reference implementations of common services (e.g. Google Drive, Slack, GitHub) and a *registry* site (registry.modelcontextprotocol.io) for published servers [72†L231-L239] [26†L218-L221] . Key developers include David Soria Parra and Justin Spahr-Summers (Anthropic) with community contributions (e.g. Microsoft provided the C# SDK, JetBrains supports Kotlin, etc.) [83†L99-L103] [72†L182-L193] .

Below is a summary of official SDK/tool implementations:

Tool/SDK	Language	GitHub Stars	License
ModelContextProtocol (Spec)	TypeScript	9.2k★ [72†L231-L239]	MIT [69†L228-L232]
MCP “servers” (directory)	TypeScript	90.2k★ [72†L231-L239]	MIT
Python SDK	Python	24.2k★ [73†L241-L249]	MIT
TypeScript SDK	TypeScript	13.3k★ [73†L241-L249]	MIT
C# SDK	C# (.NET)	4.5k★ [73†L253-L260]	MIT
MCP Inspector	TypeScript	10.8k★ [73†L253-L260]	MIT
Kotlin SDK	Kotlin	1.45k★ [73†L323-L327]	MIT
Rust SDK	Rust	3.90k★ [73†L399-L402]	MIT
PHP SDK	PHP	1.60k★ [73†L369-L373]	MIT
Ruby SDK	Ruby	0.905k★ [73†L353-L360]	MIT
... and more	...	...	MIT

All official code is MIT-licensed [69†L228-L232] . Many third-party libraries and adapters also exist (e.g. community MCP servers for Terraform, Kubernetes, etc). For reference, some early adopters and deployments include Google Drive, Slack, GitHub, Postgres servers (provided by Anthropic) [83†L54-L59] ; enterprise users like Block and Apollo GraphQL (integrated in production) [83†L60-L63] ; and tools vendors (Notion, Stripe, Hugging Face, Postman) offering MCP endpoints [13†L58-L63] [83†L54-L59] .

Protocol Architecture and Components

MCP defines a **client-server** architecture (see Figure below): the *Host* is an AI application (e.g. an assistant or IDE plugin) which instantiates one or more *MCP clients*. A *MCP server* is a service process exposing tools, resources or prompts. Tools are callable functions (like APIs) with defined input/output schemas; Resources

represent data sources; Prompts are reusable text templates [46†L209-L218] [45†L139-L146] . In operation, a client first *discovers* a server’s capabilities, then *calls* chosen tools by name, and the server returns results.

【16†embed_image】 *Figure: MCP as a “USB-C” connector. AI hosts/agents (left, e.g. Claude, IDEs) connect via MCP clients to diverse external Tools and Data Sources (right) through a unified MCP interface [15†L29-L36] .*

Communication is JSON-RPC 2.0 based [17†L133-L142] [69†L254-L262] . On each request, clients include a `_meta` field carrying protocol version and client metadata. For example, the client calls `server/discover` (JSON-RPC) to retrieve supported `tools`, `resources`, `prompts`, and version info. Subsequent calls use methods like `tools/list`, `tools/call`, `resources/get`, etc. Payloads include fields like `name` and `arguments`; responses include fields like `resultType` and `content` (often an array of `{type:text, text:"..."}`) [78†L588-L597] [78†L614-L623] . (See “Sample APIs” below for examples.)

MCP supports two transport layers: **STDIO** for local clients (processes communicating via stdin/stdout pipes) and **HTTP** (long-lived SSE or chunked streams) for remote servers [18†L174-L183] [46†L224-L232] . Over HTTP, clients send a POST and use SSE for server-push notifications (and back-channel logging) [18†L174-L183] . By default, all messages go over TLS/HTTPS; servers typically act as OAuth2 resource servers for authorization [63†L65-L69] .

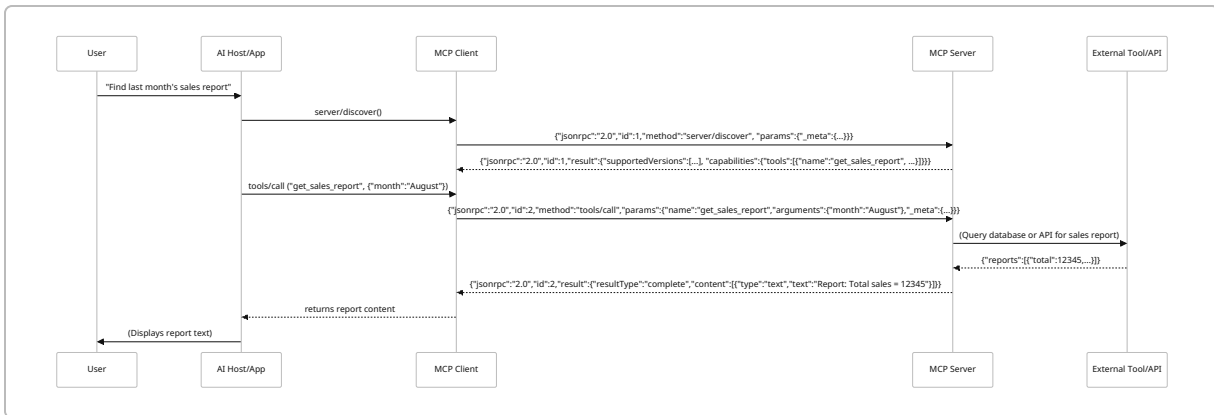
Crucial protocol aspects include:

- **Discovery:** `server/discover` (JSON-RPC) returns a list of tools/resources/prompts, each with a JSON schema for inputs/outputs [78†L435-L444] [78†L454-L463] .
- **Versioning:** The server advertises supported MCP versions, and clients must use matching version fields [77†L317-L323] [77†L335-L343] .
- **Tool Invocation:** Client calls `tools/call` with `name` and `arguments`; server executes and returns a `resultType` plus `content` or data objects [78†L588-L597] [78†L614-L623] .
- **Elicitation/Sampling:** Clients have primitives like *elicitation* (server may request clarification) and *sampling* (server asks model to generate values) [17†L244-L252] .
- **Notifications:** Servers may send async events (e.g. updated tool lists) via `tools/list` with no `method` (notify) [18†L274-L282] .
- **Authentication:** The spec uses OAuth 2.1. Clients must register (dynamic client registration) and use the `mcp` scope. Latest spec requires validating the `iss` token field to trust origins [12†L131-L139] .

Overall, MCP abstracts tool interactions into standard JSON schemas and RPC calls. It enforces *no semantic rules* on the content (e.g. model prompts), leaving security and policies to the host environment [63†L65-L69] [11†L136-L143] .

Example Workflows and Sequence Diagrams

A typical MCP workflow has these steps: (1) **Discover** tools, (2) **Select/Invoke** a tool, (3) **Return** results, (4) **Continue** (possibly choose another tool or finish). For example:



In this sequence, the AI host prompts the user and directs the MCP client to discover and call a tool. The client sends JSON-RPC to the server, which fetches data (from a database or service) and returns it. The host then presents the result to the user. Multiple rounds (multi-step reasoning) can occur within one session, as the client retains context across calls [69†L254-L262] .

Figure: The above diagram simplifies a common MCP usage: the AI “agent” is the Host, the MCP Client handles messaging, and the MCP Server exposes tools (e.g. database queries, web searches).

Other workflows might involve **event-driven** or **edge scenarios**. For instance, an IoT device (ESP32) could act as an MCP server exposing a tool `read_temperature` . An AI host could call this tool via MCP to incorporate real-time sensor data in reasoning. Conversely, the ESP32 could be an MCP client that calls remote tools (e.g. sending images to a vision model and getting a label). This bidirectional capability allows even constrained devices to be part of the AI ecosystem [20†L70-L74] .

Sample APIs and Data Schemas

MCP messages follow JSON-RPC 2.0. Every request has form:

```
{ "jsonrpc": "2.0", "id": X, "method": "server/discover", "params": { "_meta": { "io.modelcontextprotocol/protocolVersion": "2026-07-28", ... } } }
```

For example, a discovery request might look like [77†L317-L323] :

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "server/discover",
  "params": {
    "_meta": {
      "io.modelcontextprotocol/protocolVersion": "2026-07-28",
      "io.modelcontextprotocol/platformName": "myAIApp",
      ...
    }
  }
}
```

```

    }
  }
}

```

The server response includes its version info and lists of available primitives [\[77†L335-L343\]](#) . For example, a `tools/list` call might return:

```

{
  "jsonrpc": "2.0",
  "id": 2,
  "result": {
    "resultType": "complete",
    "tools": [
      {
        "name": "calculate_sum",
        "inputSchema": {"$schema": "http://json-schema.org/", "type": "object",
"properties": {"a": {"type": "number"}, "b": {"type": "number"}}, ...},
        "outputSchema": {"$schema": "...", "type": "object", "properties": {"sum":
{"type": "number"}}}}
      },
      ...
    ]
  }
}

```

Each tool definition includes a JSON Schema for its inputs and outputs. When invoking a tool, the client sends a `tools/call` request. For example [\[78†L588-L597\]](#) [\[78†L614-L623\]](#) :

```

{
  "jsonrpc": "2.0",
  "id": 3,
  "method": "tools/call",
  "params": {
    "name": "weather_current",
    "arguments": { "location": "San Francisco" },
    "_meta": { "io.modelcontextprotocol/protocolVersion": "2026-07-28", ... }
  }
}

```

The server processes the request (e.g. querying a weather API) and returns a result:

```

{
  "jsonrpc": "2.0",
  "id": 3,

```

```

    "result": {
      "resultType": "complete",
      "content": [
        { "type": "text", "text": "Current weather in San Francisco is sunny,
22°C." }
      ]
    }
  }
}

```

These examples show the minimal structure: every request/response is JSON-RPC, with parameters under `params` and results (including tool outputs) under `result` [78†L588-L597] [78†L614-L623]. Tools and resources are fully described by JSON schemas in the discovery phase [78†L454-L463], enabling automated validation and UI generation on the client side.

Integration with LLMs, Agents, and Edge Devices

MCP is explicitly designed to work with LLM-based agents and traditional ML models alike. AI **hosts/clients** such as Claude, ChatGPT (via plugins/developer mode), Cursor, or custom agents can use MCP to fetch live data and invoke services. For example, ChatGPT’s “developer mode” (Plus users) now supports connecting to MCP servers: one user reports successfully creating several MCP connections and querying them with JSON payloads [35†L20-L24]. OpenAI’s new Agent Plugins (2026) standardizes packaging of such tools (including an `mcp.json` manifest) to enable multi-agent distribution [34†L181-L188]. LangChain users have also built MCP integrations – e.g. a community project shows a LangChain agent calling a local MCP file-server to get file metadata [48†L189-L193]. In practice, any ML model or service can be wrapped in an MCP server. For example, a custom server could expose an image classifier or weather model via MCP; the LLM client would treat it like any other tool, sending inputs (like an image or location) and receiving results as structured output.

In **edge scenarios**, MCP enables low-power devices to participate in the AI ecosystem. As an example, an ESP32 microcontroller (with MicroPython or C++) can host an MCP server for attached sensors, while also acting as an MCP client to reach cloud tools. The “ESP-Claw” project demonstrates this: an ESP32 supports MCP, allowing agents to discover its functions and enabling the microcontroller to call other MCP tools [20†L70-L74]. Slides from a demo show an ESP32 voice assistant using MCP to connect local sensors to AI models [74†L66-L72]. In summary, MCP decouples the physical or on-device context from the AI logic, so even tiny devices can interact with LLMs and services.

Security, Privacy, and Threat Models

MCP relies on standard web security but adds novel concerns. On one hand, all communication is typically over HTTPS with OAuth 2.1 tokens. The spec treats each MCP server as an OAuth 2.1 resource server [63†L65-L69]. In the latest spec, servers must return a token “issuer” (`iss`) and clients must validate it, and dynamic client registration is required (with explicit `application_type`) to prevent spoofing [12†L131-L139]. However, MCP itself does **not** dictate fine-grained access control on tools – hosts or gateways must implement allow-lists, auditing and policy enforcement. For example, Maxim AI’s analysis notes that while MCP uses OAuth scopes, per-call authorization policies are *outside* the protocol’s scope [63†L65-L69].

The main threat model arises from MCP's openness and flexibility. Research (Yang et al., 2026) finds that MCP's RFC-style design (with only ~20% of clauses mandatory) creates a “tightrope” where mismatches between clients/servers can be exploited [43†L130-L137] . They identified *compatibility-abuse attacks*: e.g. prompt-injection via malicious tool descriptions, or denial-of-service by exploiting optional behaviors [43†L70-L79] [43†L130-L137] . Practical advice includes implementing thorough input validation and conformance tests (their team built multi-language testers) to avoid subtle bugs. The move to a stateless protocol also opened new vectors: if a load-balancer or proxy naively trusts the `mcp-session-id` header (now removed) or routes based on headers alone, an attacker could inject requests out-of-context. As Nordic APIs warns, “if a proxy bases authorization purely on an MCP header, an attacker could fake it” – servers should always verify the actual token/body, not just headers [11†L136-L143] .

Privacy concerns depend on deployment: sensitive data sent in queries or results will pass through MCP endpoints. Hosts must ensure that user consent is obtained before sending personal data to any model. No encryption beyond TLS is built-in; developers should also consider logging and encryption of stored context. In short, MCP provides secure channels (OAuth + TLS) but delegates all higher-level access control to the application layer. A forthcoming paper at MobiArch 2025 even proposes adding **dual cryptographic signatures** to MCP messages for end-to-end integrity [42†L91-L94] .

Scalability, Performance, and Deployment

The July 2026 stateless redesign significantly improved scalability. Because no session state is kept, *any* server instance can handle any request, allowing horizontal scaling via ordinary load balancers [11†L72-L77] . As noted by Nordic APIs, this eliminates the “pinning” issue where earlier MCP required a client’s requests to hit the same server replica. Stateless MCP can scale to large loads like a typical RESTful service [11†L72-L77] .

Performance-wise, MCP’s overhead is quite low. Real-world benchmarks (e.g. Maxim AI’s Bifrost gateway) report about **11 microseconds** added per request at 5000 RPS [63†L28-L33] . This suggests that JSON encoding/decoding and RPC framing are not a bottleneck for most use cases. Of course, actual latency will be dominated by the invoked tool (e.g. a database query or ML inference). Persistent connections (HTTP/SSE) keep overhead minimal for multi-step dialogs. In high-throughput environments, servers should implement typical microservice optimizations (connection pools, caching tool lists, etc.).

Deployment options are flexible (see Figure below). MCP servers are just web services, so they can run as local Docker containers, Kubernetes pods, cloud functions, or even embedded in edge devices. For example, a **Local Container** (developer laptop or on-prem VM) is easiest to set up (low effort, no enterprise governance) [63†L112-L119] . A **Self-Hosted** in-VPC model gives moderate control (governance depends on product) [63†L116-L119] . At the extreme, **Air-Gapped** deployments allow full data control (but at high effort) [63†L119-L122] . Conversely, a **Managed SaaS** MCP service (vendor cloud) offers plug-and-play ease (minimal ops) but leaves security policy to the provider [63†L121-L124] . There is also the option to implement MCP behind an existing **API Gateway** extension – this reuses existing ingress infrastructure but may lack native MCP semantics (tools, prompts) [63†L123-L126] .

[64†embed_image] *Figure: MCP gateway deployment models vs. governance depth. Local containers offer minimal governance overhead, SaaS is easiest to operate, while air-gapped on-premises yields maximum control* [63†L22-L30] .

In practice, organizations treat MCP servers like any microservice: they use CI/CD pipelines to deploy them (Docker/Kubernetes), configure TLS and OAuth, and monitor with standard tooling (logs, Prometheus, OpenTelemetry). Some ecosystems offer specialized MCP gateways (e.g. Maxim AI's Bifrost) to centralize policy, rate-limiting, and audit logs for many MCP servers [63†L28-L33] . Edge deployments (e.g. on an ESP32) will typically use the STDIO or lightweight HTTP mode and may run on MicroPython or native C++ with a minimal MCP library.

Interoperability with Other Standards

MCP coexists with other AI/data integration approaches. It complements **Retrieval-Augmented Generation (RAG)** by providing *active* data access rather than just retrieval. As one analysis notes, RAG “allows models to get relevant documents... but cannot take actions” – MCP goes further by enabling LLMs to *act* on systems (e.g. updating a CRM) [69†L288-L296] . In fact, MCP is often used in hybrid solutions: an agent may use RAG to fetch static info, then MCP to execute tasks.

In the agent ecosystem, MCP works alongside standards like OpenAI's Tool/Function calling. OpenAI's *Agent Plugins* spec (2026) essentially packages up instructions for tools and includes an MCP “plugin” manifest (`mcp.json`) so that tools described by an OpenAPI schema can be treated as MCP servers [34†L181-L188] . Many users compare the two: MCP is the *protocol* (like HTTP for APIs), whereas Agent Plugins define the *packaging* of tools (akin to how UML packages). Regardless, major LLM platforms (ChatGPT, Claude, AWS AI) are adopting MCP at the runtime level [34†L181-L188] [35†L20-L24] .

For developer frameworks, LangChain and similar libraries are starting to support MCP. Community examples exist where a LangChain agent calls an MCP server as a custom tool [48†L189-L193] . Webhooks and gRPC can be used as transports for MCP (e.g. a gateway translating MCP to internal gRPC calls), but the base spec does not depend on them. One theoretical proposal suggests using gRPC with protobuf (like CORBA) to add binary efficiency and streaming, but this is outside the official spec [38†L400-L408] . In summary, MCP plays nicely with HTTP/REST infrastructure, and there are emerging adapters for function-calling and agent frameworks.

Developer Tooling, SDKs, and Testing

A rich ecosystem of **SDKs** helps developers implement MCP clients and servers. The official multi-language SDKs (Python, TypeScript, etc.) handle JSON-RPC framing, request/response validation, and SSE streaming. For example, Python's `mcp.client` and `mcp.server` libraries let you annotate functions as tools. All SDKs provide methods for discovery and calling by name, and they validate inputs against the JSON schemas provided by the server.

For testing and debugging, the ModelContextProtocol org provides **MCP Inspector** (TypeScript, 10.8k stars [73†L253-L260]) – a web-based GUI that connects to a running MCP server, lets you exercise tools interactively, and view logs and schemas. Developers can also use standard HTTP clients (curl, Postman) by sending the JSON-RPC bodies to the MCP endpoint. Unit tests often mock the `Client` interface or run a local server. OpenTelemetry and structured logging can be integrated through the `_logging` channel.

The protocol's open nature has encouraged additional tools. The MCP Registry and GitHub servers list help discover connectors. Community-contributed tools include CLI generators for `mcp.json` manifests,

Kubernetes operators for MCP servers, and security scanners. Overall, testing strategies focus on *conformance*: Yang et al. emphasize automated checks that clients/servers adhere to all must/optional clauses [43†L70-L79] [43†L130-L137] . For robust systems, teams often deploy a proxy gateway (like Bifrost) to centrally apply ACLs, logging, and quotas on MCP traffic.

Use Cases and Case Studies

MCP is already used in real-world AI systems for a variety of scenarios:

- **Enterprise Productivity:** Anthropic released reference MCP servers for popular services: Google Drive, Slack, GitHub, local Git repos, Postgres, and even Puppeteer (headless browser) [83†L54-L59] . In one demo, an AI agent could search Slack channels, read Google Docs, or commit to GitHub via standard MCP calls. Notably, Notion built an MCP server to allow AI to search and update notes; Stripe built one to automate payment workflows; Postman uses MCP for smart API testing; and Hugging Face has servers for model/dataset search [13†L58-L63] [83†L54-L59] .
- **Software Development:** GitHub and VS Code integrations allow tools like code summarizers and reviewers to use MCP. For example, a ChatGPT extension could call an MCP server to fetch code snippets or run linters. Key companies have adopted MCP here: Microsoft's Copilot (via Visual Studio Code) and GitHub's agentic tools support MCP, enabling better context passing around coding tasks [63†L56-L64] .
- **Business Automation:** Block (Square) uses MCP internally, highlighting that “open technologies like MCP are bridges connecting AI to real-world applications” [83†L60-L63] . In finance and CRM, an AI assistant could use MCP to pull customer records, update databases, or schedule meetings. Security provider Keyfactor describes using MCP for **PKI automation**, e.g. letting AI manage certificate lifecycles [52†L125-L133] . A Keyfactor engineer summarized: “Generative AI can talk, and agentic AI can do, using the MCP” [52†L125-L133] .
- **IoT and Robotics:** Embedded use cases include home automation and robotics. For instance, an AI model running in the cloud could MCP-call an on-device server to “turn on the lights” or “adjust a thermostat”. Conversely, devices can report telemetry via MCP back to an LLM agent. The ESP32 voice assistant demo (slide above) shows how a microcontroller with sensors can become part of the agent network [20†L70-L74] [74†L66-L72] .
- **Data Analytics:** MCP can connect AI to business intelligence tools. For example, an AI consultant could query an MCP-enabled BI system to analyze sales trends or inventory levels, then generate a report. The protocol's flexibility means it applies to many domains (healthcare imaging, scientific workflows, etc.), wherever LLMs need live data or control over systems.

Case Study – Sentry: As a concrete example, Sentry (the error-monitoring platform) provides an MCP server (at *mcp.sentry.dev*) so that LLMs can query logs and manage issues programmatically. The Sentry MCP lets an AI assistant search recent errors or create Jira tickets based on logs [28†L1-L6] . This illustrates how MCP turns a passive monitoring system into an “agentic” tool.

Limitations, Open Problems, and Future Directions

While powerful, MCP is still evolving. Key limitations and open issues include:

- **Complexity and Compliance:** MCP's flexibility comes at a cost: many protocol clauses are optional, which can lead to interoperability bugs. The Yang et al. study found thousands of issues in MCP SDKs and servers, warning that “compatibility-abusing attacks” (e.g. injection, DoS) are real threats if implementations diverge [43†L70-L79] [43†L130-L137] . Better compliance testing and perhaps a stricter conformance profile could help.
- **Security Gaps:** As noted, per-request authorization (tool whitelisting, context filtering) is not defined by MCP and must be built separately. Without careful host design, an LLM might inadvertently access or corrupt data. Future work may include formal threat models and defense patterns (e.g. mandatory sandboxing, signature verification [42†L91-L94]).
- **State Management:** MCP is essentially stateless now; it doesn't define global state or long-term memory. Applications needing persistent agent memory must implement it externally. Research could explore stateful extensions or standardized “memory” resources.
- **Limited Media Handling:** The protocol is optimized for JSON/text data. Handling streaming media (audio/video) or very large binary data requires additional infrastructure (e.g. pre-signed URLs). Extensions or best practices for multimodal data over MCP are still being developed.
- **Lack of Semantic Awareness:** MCP treats tools as untyped RPC. Unlike gRPC or CORBA, there is no built-in IDL or language-agnostic object passing beyond JSON. Some propose adding Protocol Buffers or IDL layers for performance and stricter typing [38†L400-L408] , but this is still conceptual.
- **Ecosystem Maturity:** Though growing fast, MCP is newer than some standards (e.g. REST, GraphQL). It lacks formal governance and widespread third-party libraries (e.g. for C/C++). The Linux Foundation “Agentic AI” working group (Dec 2025) is a step towards standardization, but more open collaboration will strengthen it.
- **Future Research:** Active areas include formalizing **Agent Plugins** integration, exploring hybrid RAG/MCP agents, and AI-driven policy enforcement. The concept of *delegated AI* (where models grant limited access to tools) could use MCP as the foundation. Researchers may also study performance tuning, cache consistency, and scalability in multi-hop MCP workflows. Finally, advances in on-device ML and networking may enable fully decentralized MCP networks (multiple LLMs and tools coordinating without a central host).

Conclusions and Next Steps

The Model Context Protocol establishes a robust foundation for *agentic AI*: enabling LLMs and other models to interact dynamically with external systems. Its official specification, community adoption, and multi-

language tool support make it a practical choice for extending AI capabilities [83†L21-L25] [72†L174-L178] . For students or developers looking to experiment, the recommended next steps are:

1. **Build a simple MCP server.** For example, in Python you can use a library like `fastmcp` to expose a trivial tool (say, a sentiment analyzer). *Example:*

```
# mcpsrv.py
from fastmcp import MCPServer, Tool

server = MCPServer()

@Tool(name="sentiment", description="Classify text sentiment")
def sentiment_analysis(text: str) -> str:
    return "Positive" if "good" in text else "Neutral"

server.run("localhost", 8000)
```

Running this will start an MCP server at `http://localhost:8000` with one tool (`sentiment`).

1. **Connect an MCP client.** In Python, use the MCP client SDK to discover and call the tool:

```
from mcp.client import MCPClient
client = MCPClient("http://localhost:8000")
tools = client.discover().tools
result = tools.sentiment.call(text="This product is good")
print(result) # Expect {"sentiment": "Positive"}
```

This code initializes an MCP client, discovers available tools, and invokes the `sentiment` tool with a test string. The printed result shows the tool's output.

1. **Experiment with an LLM host.** Deploy an LLM (e.g. run Claude Desktop or use ChatGPT dev mode) and register your MCP server as a tool. Instruct the model (via a prompt) to use `tools/sentiment` when analyzing text. For instance, ask: "Use the *sentiment* tool to evaluate 'I love programming'." The model should call your MCP endpoint and return "Positive".
2. **Iterate and Extend.** Add more tools (e.g. a calculator or a dummy ML model), try different transports (stdout vs HTTP), and examine how your client and server interact (logs). Use **MCP Inspector** or a network sniffer to see the JSON-RPC messages. Test authentication by requiring an OAuth token on the server.

These steps give hands-on insight into MCP's workflow. For a demonstration, one could combine the server above with any local LLM or API-callable model. The code snippets above illustrate the minimal MCP server/client needed. From here, students can explore additional tools, integrate with real data sources (databases, web APIs), and evaluate how the model uses these new capabilities.

Recommended Reading: The official MCP docs (modelcontextprotocol.io) and Anthropic’s blog provide the authoritative spec [83†L21-L25] [15†L29-L36] . Key analysis and case studies from industry (e.g. Maxim AI’s posts [63†L22-L30] , Oracle [44†L51-L60] , Keyfactor [52†L125-L133]) give practical insights. Academic works (Yang et al. 2026 [43†L70-L79]) highlight security considerations. Together, these resources will help build a robust understanding of MCP for any AI/ML project.

Sources (prioritized): Official Anthropic and ModelContextProtocol documentation [83†L21-L25] [15†L29-L36] ; GitHub MCP repos [72†L174-L178] [73†L241-L249] ; industry analyses and blog posts (e.g. Maxim AI [63†L22-L30] , Oracle [44†L51-L60] , Keyfactor [52†L125-L133] , Themenglab [20†L70-L74]); security research (Yang et al. arXiv 2026 [43†L70-L79]); as well as community guides and examples [35†L20-L24] [48†L189-L193] . These were used to compile the above report.
